

From Mined Fixes to Gated Repair: Closed-Loop Defect Synthesis for CIRCT

Abhinav Venkata Kota, Adithya Jillellamudi, Priyesh Shukla

International Institute of Information Technology, Hyderabad

{abhinav.kota, adithya.jillellamudi}@students.iiit.ac.in, priyesh.shukla@iiit.ac.in

Abstract—Software bugs in hardware compilers like CIRCT risk injecting silent defects into physical chips. AI repair agents can fix known issues, but they are constrained by the slow trickle of human-reported bugs. We present a closed loop, built on the CHIA agent framework, that mines historical CIRCT commits to generate its own defect supply: a model learns the mistake pattern behind each of 187 past fixes and synthesises novel, bug-exposing inputs. The loop offloads building, judging, test reduction and deduplication entirely to deterministic tools, hands patch generation to CHIA’s existing repair chain, and reserves human effort exclusively for final filing approval. It restores the compiler’s internal assertions, compiled out of CHIA’s published image, in 536 of 555 objects, so defects become observable where none could fire before. In pilot deployments the loop reproduced a live open CIRCT issue from its first seed and withheld it as a duplicate, at a total inference cost of about ten US dollars across all model calls.

I. INTRODUCTION

CIRCT is an open-source compiler for hardware. A designer writes how a circuit should behave, in FIRRTL (the form the Chisel hardware language produces) or in SystemVerilog, and CIRCT lowers it to Verilog, which synthesis and layout tools turn into a chip. It is built on MLIR, the LLVM project’s compiler framework, and inherits LLVM’s way of testing: a regression test carries the command to run and the output to expect, run by a tool called `lit`. A bug that crashes CIRCT is an inconvenience, seen and worked around; one that makes it emit wrong Verilog is not: the output is well formed, passes the tools below it, and can reach fabrication unnoticed.

CHIA [1] is the agent framework this hackathon is built on. Its CIRCT issue solver chains six phases, from assessing an issue to reproducing, fixing, verifying against the whole test suite, repairing regressions and writing the pull request; on 16 CIRCT issues it fixed five, three of them merged.

Its supply is the constraint. Over the past year CIRCT received 82 bug-labelled issues and closed 65; what does not drain is the backlog of 101 open ones, at a median age of 911 days. The scarce resource is maintainer attention, and an agent that empties the queue faster creates none. What would help is a bug that arrives already minimised, already checked against what is known, already carrying a candidate fix, from a source that files nothing else. Past fixes are the unspent information: a commit that corrects one wrong assumption inside a pass usually leaves the same assumption at other sites, and CIRCT ships no fuzzing infrastructure to find them.

This paper describes a loop that manufactures its own supply and then withholds most of it.

II. THE LOOP

Fig. 1 names the stages and marks the ones the agent runs; tools decide everything else. One model serves every agent stage, Gemini 3.8 Flash through CHIA’s Vertex backend, and no stage can build a client for it unless one environment variable is set for that run, so a test or a dry run cannot reach the model.

Read the seed (1). A seed is one of 187 CIRCT commits from the last two years that look like bug fixes, with its diff and the test its author added or changed. The agent reads both, states in one sentence the root-cause class the fix corrected, and names other sites where the same assumption still stands.

Write probing inputs (2). The seed’s own test decides the language, because its command line names the tool through which the bug was reachable: `circt-opt`, `firtool` or the SystemVerilog front end. About 127 of the 187 seeds sit in code no FIRRTL input can reach, so a FIRRTL-only generator would miss most of the corpus. The agent reads the compiler’s source and writes each candidate through tools of its own, and runs no command.

Build (3). CIRCT compiles against one pinned commit of LLVM and MLIR, and Section III explains how the loop avoids building those from source. The build compiles in CIRCT’s assertions, the internal checks its own authors wrote, which stop the compiler the moment an assumption breaks.

Judge (4). An oracle is the rule that decides whether an input found a bug. The primary one admits no argument: the compiler crashed, or an assertion fired. A second runs the same design through CIRCT’s simulator and through Verilator, and treats disagreement as suspicious; it reports only, because the two disagree routinely over uninitialised and reset signals.

Reduce (5). A crashing input is usually large and mostly irrelevant to the crash. `circt-reduce`, which CIRCT ships, deletes parts of it and keeps each deletion while the failure survives; a textual reducer takes over where that tool dies.

Triage and report (6). Many failures are one failure, and deciding which are the same is mechanical: a fingerprint is the assertion text with its source location, or, for a crash, the symbolised frames left once the crash handler’s prologue is stripped. Survivors are matched against a local mirror of CIRCT’s open and closed issues and against commits made since the build point. The agent writes the report, and no gate reads what it concluded.

Repair (7). CHIA’s phase chain runs unaltered but for one branch selecting the backend the rest of the loop uses and an

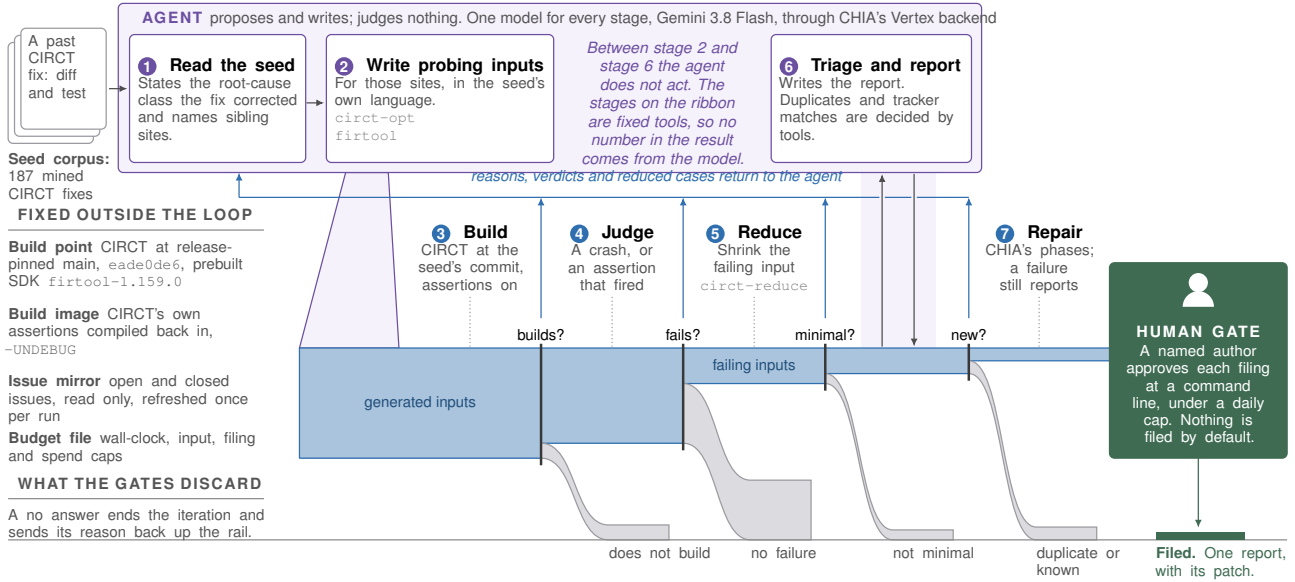


Fig. 1. Seeded CIRCT bug loop. The agent holds the violet band and acts three times only, at stages 1, 2 and 6; the blue ribbon below it is fixed apparatus, so no number in the result comes from the model. The ribbon is the candidates in flight, pinched at each gate by a yes-or-no question; the share answering no peels onto the floor. Widths are illustrative.

TABLE I
WHAT WE MEASURED ON THE IMAGE THE LOOP RUNS

Measurement	Result
<i>Building the past cheaply</i>	
Fix-shaped commits mined from 24 months	187
Of those, seeds with an exactly matching prebuilt SDK	171
Pin windows with no matching release, worst 16 days	16 of 63
<i>Letting the compiler complain</i>	
Compile commands carrying -UNDEBUG	778 of 778
Object files carrying the assertion handler again	536 of 555
CHIA's whole test gate, assertions on and then off	1,119 pass, 0 fail, identical
What the assertions cost: wall clock, bytes, per probe	18%, 25%, 4%
Tests lit discovers, once its source root is the SDK	1,390, 0 errors

entry point taking a local report in place of a GitHub issue. Repair is restricted to crashes and assertion failures, where nobody has to argue about what the right answer was.

The gates. Between stages sit questions with two answers, the bars that pinch the ribbon in Fig. 1. A no ends the iteration and returns its reason to the agent with the verdicts and reduced cases. The last is a gate of four questions, each answered by a tool: does the failure reproduce in a fresh process on another worker, is the case minimal, is the input valid, and is it new. The third is three parse-and-verify commands, and a clean rejection makes the input the loop's own fault rather than the compiler's. Then the gate is a person: a named author approves every filing at a command line, under a cap on filings per day, and nothing is filed by default.

III. MEASURED PRECONDITIONS

The loop has to build CIRCT at arbitrary points of its history, and the compiler has to be able to complain. Neither held when we started; Table I is what we measured on the image the loop now runs.

Building the past cheaply. CIRCT records the exact LLVM commit it expects, its pin; a build against any other fails,

and compiling that LLVM costs hours per seed. CIRCT also publishes an SDK of prebuilt LLVM and MLIR libraries carrying its own pin, and where the two pins match CIRCT rebuilds in minutes: 171 of the 187 seeds do.

The same mechanism fixes where the loop runs: not the newest commit of CIRCT's main branch, whose pin has no published release today, but release-pinned main, the newest commit whose pin has one, which for this image is 8 commits and 4.8 days behind the head. A long lag mechanically lowers the filing rate, because more candidates come back already fixed.

Letting the compiler complain. Optimised builds delete assertions by defining the macro NDEBUD, and CHIA's published CIRCT image is such a build: its binaries hold no reference to the assertion handler, so a loop whose oracle is a failed assertion would find nothing under it, whatever the agent wrote. One compiler flag, -UNDEBUG, puts them back, and the image the loop runs is that build at release-pinned main, with the flag in every compile command and the handler back in all but 19 of its 555 compiler objects, which assert nothing.

Whether the restored checks fire on CIRCT's own tests is what the oracle rests on, so we built the image twice, differing in that flag alone, and ran CHIA's whole test gate in both: the two runs pass the same 1,119 tests and fail none. No test passes with assertions compiled out and fails with them compiled in, so a probe that trips an assertion trips it because of what the probe does, and Table I prices the flag.

One defect had to be repaired first: at CIRCT commits after mid-2026 a test directory's configuration file raises while lit is discovering tests, and CHIA's exclusion of that directory is applied after discovery, so the gate discovered nothing. Pointing the build at the SDK as its MLIR source root, one flag, restores discovery. The binary still calls itself an optimised build, so

every report names the flag.

Tested on real failures. The oracle and its fingerprints were calibrated on real output: six past CIRCT bugs, rebuilt at the commit before their fix and recorded with input, argument vector and error output, in three classes across two SDK releases, 14s to 601s of build each. Two defects in the fingerprinting rules appeared only there: the name normaliser erased every frame in an anonymous namespace, which is where CIRCT’s passes live, and the scope test read an inlined frame instead of the physical one, putting three of the four real bugs outside the repair stage’s reach. Both are fixed.

IV. CONTROLS

The experiment the loop embodies asks whether reading a past fix makes an agent better at finding new bugs than mutating the same tests at random. Two arms share the harness, oracles, reducer and gate, and only the generator differs: the seeded arm is stages 1 and 2, and the mutation arm rewrites the same seed tests syntactically without seeing the diff or the root-cause class, following Mut4All’s design for mutators synthesised by a language model [2]. Both arms spend the same budget, a fixed window of wall-clock seconds, one after the other on the same cluster at the same concurrency, and the artefact renders one headline: distinct bugs a CIRCT maintainer confirmed by labelling, commenting on or fixing them. Our oracle agreeing with itself produces a candidate, not a bug.

Pre-registration. Every threshold lives in one budget file, and a run is registered by an annotated git tag on the commit that lands that file; the loop refuses a file no tag names and a mutator set committed after the tag, so no threshold can be chosen after the fact. Safety caps bound damage rather than effort: 2,000 generated inputs per arm per day, 3 filings per day and 10 per run, 60 seconds and 4 GiB per probe, and a hard cap on model spending that stops both arms. A run with a positive filing cap does not start until a human has posted the method to the project’s forum, and issues marked as good first issues are refused.

Deduplication. Two candidates are one bug when a single primary fingerprint is equal. On a hand-labelled set of 22 candidate pairs the rule collides on 1 of the 11 distinct pairs and falsely merges 1 of the 11 duplicate pairs; both rates are printed beside the headline, since a collision inflates that count and a false merge deflates it.

Known issues. A mirror of the tracker answers whether a candidate is already reported. GitHub stops paginating an issue listing at its hundredth page, which the loop met as an error and not as truncation, so the mirror walks newest first, then oldest first, and unions the two: the whole tracker, 2,652 issues, 1,795 closed and 857 open, costs 198 requests and 200 seconds.

Contamination. CIRCT’s history sits in the training data of any model we can use, so the agent may reproduce a fix it memorised. Every candidate is screened against fixes made after its seed, at the level of the symbols the seed’s fix touched: that screen flags 77% of the corpus where the same screen at

TABLE II

EVERY LIVE MODEL RUN, FROM THE LOOP’S LEDGER; THE LAST COLUMN IS BILLED OVER WHAT THE GUARD AUTHORISED BEFORE THE FIRST CALL

Run	turns	seeds	probes	fired	filed	USD	billed / authorised
Mutator set 1 (22 in, 11 kept)	1	–	–	–	–	0.49	0.37
Mutator set 2 (25 in, 25 kept)	1	–	–	–	–	0.49	0.26
Pilot 1, seeded, cap 5.00	3	1	3	3	0	7.16	19.7 to 64.1
Pilot 2, seeded, cap 3.00	2	2	0	0	0	0.81	0.50 to 0.59
Pilot 2, mutation	0	2	5	0	0	0	–
Pilot 3, seeded, cap 3.00	1	1	0	0	0	0.64	0.24

file level flags 96%. A real crash is a real crash, so the flag qualifies a candidate rather than excluding it.

V. RESULTS

Five live runs have been made and Table II is all of them, read from the loop’s own database and ledger; the ledger prices the SDK’s own token counts at Google’s published rates.

The restored assertions are reachable, on the first seed. Pilot 1 read its first seed, a fix in the SystemVerilog lowering, wrote three probes, and all three tripped the same check, `isa<To>(Val) && "cast<Ty>() argument of incompatible type!"` at `Casting.h:566` in `NetOpConversion::matchAndRewrite` of `MooreToCore.cpp`, on a three-operation case, a `moore.net` of an open array type; each reproduced in a fresh process on another worker, and twenty reduction attempts removed nothing. The oracle Section III restored found a real, reproducible failure with a three-line reproducer at the first attempt.

Withholding works, and the tool outranks the model. The mirror matched all three to open issue 8508 on the assertion text, and nothing was filed. The one report a model wrote argued the match was wrong; the tool’s verdict stood, by rule. A maintainer sees nothing they already know, and the model cannot promote a candidate.

The money control, measured twice. Pilot 1’s guard priced one model call per turn, but a turn carrying tools makes one call per tool iteration: its three turns made 14, 21 and 37 calls and cost 19.7, 32.3 and 64.1 times the authorised figure, USD 7.16 against a cap of 5.00; the cap still stopped the run. The guard now prices the whole loop, and a ceiling inside the backend refuses the call that would pass it: pilot 2 billed 0.50 and 0.59 of what it authorised. Spending is bounded before a request leaves the machine, within a factor of two of the bill.

The tool budget decides whether the model answers. At six tool calls a turn, both of pilot 2’s seed-reading turns spent all six reading source and returned no answer, so no probe was written; the turn that read pilot 1’s firing seed had taken fourteen. At twelve, with one extra tool-free call demanding an answer, pilot 3’s turn made thirteen calls and that answer came back empty. The cap is registered, and this is the measurement it is chosen with.

Seven defects no test could see. The first run driven end to end on the cluster met seven pre-flight defects invisible to 620 passing tests, none about the model: among them, the corpus

emptied because its date bound was the run’s start, and a check demanded the head hold a credential the design forbids it. All were fixed before a model call was made: a capped pilot is part of the method.

Under-billing, and one fact that doubled the yield. CHIA’s Vertex backend omits thinking tokens from usage; on the two synthesis turns they were 2.3% and 2.4% of the price, and an eighteen-line patch makes the ledger equal the SDK’s total exactly, 625,478 tokens on the first turn. That turn returned 22 mutators and 11 survived: ten used look-behinds Python’s regular-expression engine refuses, because the prompt named no engine. Naming it, the second returned 25 and all survived. In pilot 2 they yielded five probes in under two seconds, four of which failed to parse, the loop’s own fault; none fired.

VI. RELATED WORK

SWE-bench [3] scores agents on real GitHub issues and CHIA’s issue solver [1] applies the pattern to a hardware compiler; both draw from a queue, and we fill one. Fuzzers fill queues without repairing or withholding: CSmith [4] for C, and for MLIR, MLIRod along operation dependencies [5], Nüwa with a language model [6], DESIL on silent wrong-output bugs [7] and FLEX, 80 unknown bugs in 30 days [8]. Our mutation arm follows Mut4All’s synthesised mutators [2]. AFuzz [9] is the origin of our method, an agent hypothesising where else a reference bug’s cause could live and testing each hypothesis, 40 bugs in V8 in a month; we apply it behind a repair chain and a gate. Delta debugging [10] is stage 5’s algorithm, and we contribute its placement. Abstain-and-Validate [11], llvnharness [12] and CodeRover-S [13] each make a repair agent more selective; our gate sits downstream and discards by default.

VII. ARTEFACT AND LIMITATIONS

We release the loop as CHIA nodes and tools, so swapping the generator for the mutation arm is one node substituted. Its two halves meet at a frozen contract of seven record types and one function call, version 2.2, tested separately against recorded examples; the suite is 634 tests, 594 of which need no cluster, network or model. On CHIA’s cluster layer the loop’s five containers come up in 38 seconds and, with a recorded generator, drive 16 probes through both arms in 7.4 seconds. Four things go upstream to CHIA: the Dockerfile for the assertions-on image, its build workflow, three helpers for its CIRCT module, and the nineteen-line branch that lets its repair chain select our backend. All of it, with the seed corpus and every input, verdict, reduced case, transcript and gate decision, is at https://github.com/DevAbhinav-23/CHIA_Hackathon_2k26.

The second oracle’s harness generators emit, from one design, a CIRCT simulation harness and a Verilator testbench printing identical 112-line sequences, and one changed constant makes them disagree at a named cycle and signal.

Limitations. Model time across the five live runs totals USD 10.1, and the repair stage, CHIA’s chain, ran in none of them, because it returns no token counts and would spend outside the

ledger. The second oracle applies to 6 of the 187 seeds. The 187 rest on a subject-keyword filter, no diffs read to confirm each is a bug fix; of seven mining attempts, one seed did not fail at its parent. The contamination screen misses a sibling fixed in a commit whose subject does not name it. Calibration by re-injecting known bugs is not possible on one prebuilt image: no seed’s parent shares its LLVM pin. The image is built but not published, so a report names its content hash rather than a registry digest.

Scope. The loop does not repair wrong-output bugs, because the oracle that would find them is report-only, and it does not replace maintainers: it converts the human event from writing a bug report into reading one.

ACKNOWLEDGEMENTS

We thank the CHIA developers and the A³ hackathon organisers. *AI-assistance disclosure*. Claude (Anthropic) and Gemini (Google) were used for drafting, code generation and revision; the human authors made every claim, decision and number their own and are responsible for the whole paper.

REFERENCES

- [1] A. Cui *et al.*, “CHIA: An open-source framework for principled, agentic AI-driven hardware/software co-design research,” *arXiv preprint arXiv:2606.27350*, 2026.
- [2] B. Wang *et al.*, “Mut4All: Fuzzing compilers via LLM-synthesized mutators learned from bug reports,” *arXiv preprint arXiv:2507.19275*, 2025.
- [3] C. E. Jimenez *et al.*, “SWE-bench: Can language models resolve real-world GitHub issues?” in *Proc. Int. Conf. on Learning Representations (ICLR)*, 2024.
- [4] X. Yang, Y. Chen, E. Eide, and J. Regehr, “Finding and understanding bugs in C compilers,” in *Proc. ACM SIGPLAN Conf. on Programming Language Design and Implementation (PLDI)*, 2011, pp. 283–294.
- [5] C. Suo, J. Chen, S. Liu, J. Jiang, Y. Zhao, and J. Wang, “Fuzzing MLIR compiler infrastructure via operation dependency analysis,” in *Proc. 33rd ACM SIGSOFT Int. Symp. on Software Testing and Analysis (ISSTA)*, 2024, pp. 1287–1299.
- [6] B. Cao, W. Tong, Z. Tang, Z. Wang, H. Huang, and Y. Yan, “Nüwa: Enhancing MLIR fuzzing with LLM-driven generation and adaptive mutation,” in *Proc. IEEE Int. Conf. on Software Maintenance and Evolution (ICSME)*, 2025.
- [7] C. Suo, J. Wang, Y. Wang, J. Jiang, Q. Shen, and J. Chen, “DESIL: Detecting silent bugs in MLIR compiler infrastructure,” *arXiv preprint arXiv:2504.01379*, 2025.
- [8] Z. Sun, J. Liang, W. Wang, C. Suo, J. Chen, and F. Xu, “Interleaved learning and exploration: A self-adaptive fuzz testing framework for MLIR,” in *Proc. IEEE/ACM Int. Conf. on Automated Software Engineering (ASE)*, 2025.
- [9] J. Park and I. Yun, “Agentic fuzzing: Opportunities and challenges,” *arXiv preprint arXiv:2605.10074*, 2026.
- [10] A. Zeller and R. Hildebrandt, “Simplifying and isolating failure-inducing input,” *IEEE Trans. Software Engineering*, vol. 28, no. 2, pp. 183–200, 2002.
- [11] J. Cambroneiro *et al.*, “Abstain and validate: A dual-LLM policy for reducing noise in agentic program repair,” in *Proc. IEEE/ACM Int. Conf. on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, 2026.
- [12] Y. Zheng, C. Li, S. Li, Y. Zhang, and Z. Su, “Agentic harness for real-world compilers,” *arXiv preprint arXiv:2603.20075*, 2026.
- [13] Y. Zhang, J. Wang, D. Berzin, M. Mirchev, and A. Roychoudhury, “Fixing security vulnerabilities with agentic AI in OSS-Fuzz,” in *Proc. IEEE/ACM Int. Conf. on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, 2026, pp. 374–384.